Mini Project 1:
Multi-Client Chat Application Using Java Sockets

Project Requirements

In this mini-project, you will gain experience working with Java Sockets by implementing a group chat application using TCP (Transmission Control Protocol) and JavaFX. The application enables real-time communication between multiple clients through a central server.
Your task is to develop a server-client application that allows multiple users to connect to a central server and exchange messages in a shared chat environment.

1. System Architecture

The application follows a **Server-Client architecture**.

- **Central Server:** Acts as a distributor. It receives messages from any single client and sends them to all other connected clients.
- **Multiple Clients:** Individual applications that connect to the server's IP and port to send and receive messages.

2. Functional Requirements

2.1 Client Features

- **Authentication & Identity:** Users must enter a username before accessing the full chat interface.
- **Read-Only Mode:** If a user connects without providing a username, the application must restrict them to "READ-ONLY MODE," preventing them from sending messages.
- **Real-time Messaging:** Users can type messages into a text area and send them to the group using a "SEND" button or by hitting enter.
- **Active User Inquiry:** By entering the command **"allUsers"**, the server must return a list of all currently active clients to that specific user.
- **Disconnection:** A user can disconnect from the server by typing "end", "bye", or using a "LEAVE" button. The application should then close the connection and notify the server.
- **UI Status Indicators:** The client UI must display an "Online" status label and a visual indicator (e.g., a circle).

2.2 Server Features

- **Connection Management:** The server must be capable of accepting multiple concurrent client connections.
- **Message Distribution:** Upon receiving a message from a client, the server formats it with the sender's username and time, then distributes it to all active connections.
- **Client Monitoring:** The server UI should maintain a live **ListView** of all connected usernames.
- **Visual Distinction:** To improve readability, the server should assign random background colors to different users in the client list.
- **Logging:** The server must display a log of activities, such as "Server Started," "Waiting for Client," and "Welcome [User]" messages.

3. Technical Specifications

3.1 Technology Stack

- Language: Java.
- Network communication: Java Sockets (TCP)
- GUI Framework: JavaFX (utilizing GridPane for layout and CSS for styling).
- IDE: IntelliJ IDEA.

3.2 Server-Side Model

To handle simultaneous (i.e., concurrent) client connections, the server should use multithreading with virtual threads, allowing users to exchange messages in real time.

3.3 Network configuration parameters: The server IP address and port number

The application should load these settings from a standard configuration file (e.g., *.properties* or *.xml*) at runtime to allow for environment changes and flexible deployment without requiring code recompilation.

3.3 Model-View Decoupling (Separation of Logic and UI)

To facilitate maintainability, the application's business logic must be separated from its presentation. Consequently, the application must strictly adhere to the principle of **Separation of Concerns**. The **Model** (responsible for data processing and socket communication) MUST function independently of the **View** (the JavaFX interface). Any changes to the presentation layer should not necessitate modifications to the underlying application logic.

4. Operational Requirements

4.1 Command Line Arguments

The application must be started with specific parameters:

- **Client:**
  java TCPClient <ServerIPAddress> <PortNumber> (e.g., java TCPClient localhost 3000).
- **Server:**
  java TCPServer

5. Deliverable:

To complete the project, the following deliverables must be submitted:

- **Fully Functional Maven Projects:** Source code for TCPServer and TCPClient.
- **Artifacts:** Executable JAR files for both applications.
- **Demo Video:** A 3-minute video showcasing a walkthrough of the source code and a live demonstration of the application's features.
- **Technical Architecture (UML):**
  - Class Diagram: Detailing the software structure and relationships between classes.
  - Deployment Diagram: Illustrating the physical network nodes (Server vs. Clients) and the TCP/IP communication links.
  - Optional: Use Case diagrams or Sequence diagrams to further clarify user interactions and message flow.
- **GitHub Link:** A shared link to your GitHub repository containing the full source code, commit history, and a descriptive README.md.
- **Optional Technical Post:** A link to a blog post (e.g., Medium, Dev.to, or a personal forum) detailing the development process, technical challenges, and final implementation.

6. Expected learning outcomes:

Students will gain practical experience with TCP/UDP communication, Java I/O streams, multithreading, client-server architecture, network programming, and graphical user interface development using JavaFX.